University of Caen Normandy
UFR of Sciences
Licence 2 – Computer Science
Project 1

Project Report

Interpreter for Lindenmayer Systems (L-systems) in Java

Thomas ACAR
Zineddine GHALMI
Arthur LETE

April 18, 2026

Abstract

This study focuses on the modelling and visualisation of fractal structures through Lindenmayer systems (L-systems). The objective is to translate formal production rules into complex geometric patterns using optimised algorithms implemented in Java. Our system supports a wide range of models, from classical deterministic systems (D0L-systems) to stochastic and context-sensitive approaches. The software architecture natively integrates a two-dimensional (2D) and three-dimensional (3D) rendering engine, enabling faithful spatial modelling of biological growth processes. The results obtained demonstrate the effectiveness of these formal languages for generating sophisticated fractal geometries while ensuring smooth performance.

Contents

1 Introduction **2**
- 1.1 General Description . . . 2
- 1.2 Problem Identification . . . 2
- 1.3 Outline of the Report . . . 3

2 Project Structure **5**
- 2.1 Requirements Description . . . 5
  - 2.1.1 External File Configuration and Parsing . . . 5
  - 2.1.2 Graphical Interpretation and Rendering Engine . . . 5
- 2.2 Implemented Features . . . 6
- 2.3 Project Organisation . . . 7

3 Technical Elements **8**
- 3.1 Data Structure Description . . . 8
  - 3.1.1 Symbol Management via the Flyweight Pattern: . . . 8
  - 3.1.2 Encapsulation of Production Rules: . . . 8
  - 3.1.3 Mapping Symbols to Actions: . . . 9
  - 3.1.4 The `Turtle2D` State . . . 9
- 3.2 Algorithm Description . . . 9
  - 3.2.1 Stochastic Selection Algorithm (Random Drawing): . . . 10
  - 3.2.2 Recursive Development Algorithm (DFS): . . . 10
  - 3.2.3 Iterative Development Algorithm with Singletons (BFS): . . . 11

4 Project Architecture **13**
- 4.1 Module and Class Diagrams . . . 13
  - 4.1.1 Structure of the `turtle` Package . . . 13
  - 4.1.2 Structure of the `fractal` Package . . . 14
- 4.2 Description of Packages Used . . . 14
- 4.3 Processing Pipelines . . . 14

5 Experiments and Usage **17**
- 5.1 Use Cases . . . 17
  - 5.1.1 Configuration File Grammar . . . 17
  - 5.1.2 Automated Build/Run Script (ANT) . . . 19
  - 5.1.3 Overview of Visual Results . . . 20
- 5.2 Quantifiable Results . . . 20

6 Conclusion **22**

Chapter 1

Introduction

1.1 General Description

[1] A Lindenmayer system, or L-system, is a rewriting system used to generate strings of characters from rules. It was inspired by biology and invented in 1968 by Hungarian biologist Aristid Lindenmayer. An L-system has two main components:

1. a starting string (axiom),
2. at least one rewriting rule.

To illustrate this concept, consider the Koch curve, defined by:

- **Axiom (starting string):** F
- **Production rule:** $F \to F - F + +F - F$

In this formalism, each character has a specific graphical interpretation. For example, the character **F** instructs the system to draw a segment, while + and **-** correspond to rotations by a fixed angle (here 60°).

By applying the rewriting rule, we progress from a simple segment to increasingly complex structures:

- **Generation 0:** F (a straight segment)
- **Generation 1:** $F - F + +F - F$ (a "spike" shape. Figure 1.1)
- **Generation 2:** $F-F++F-F-F-F++F-F++F-F++F-F-F-F++F-F$ Each F from the previous step is again replaced, generating an even longer string which, once interpreted, forms a detailed fractal.

This iterative approach makes it possible to model complex self-similar structures from extremely simple rules.

1.2 Problem Identification

One of the major challenges of this project lies in efficient memory management during the generation of complex strings. To produce an L-system, the most intuitive method is to

Figure 1.1: Koch Curve – Generation 1

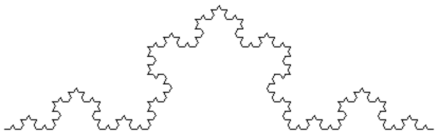

Figure 1.2: Koch Curve – Generation 4

store the axiom as a **String** and replace each character with its corresponding production rule at each new iteration.

However, in **Java**, this naive approach is extremely inefficient. Since new generations can be built iteratively (or in parallel), we must retain the old string while constructing the new one. In Java, String objects are **immutable**: each concatenation of a character to a string of length n results in the creation of a new instance and the copying of all existing content. [2] [3]

The total number of copy operations is therefore defined by the sum of integers:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Consequently, building a generation of length n through successive appends has the following complexities:

- **Time complexity:** $O(n^2)$, since each append costs $O(k)$ where k is the current length.
- **Space complexity:** $O(n^2)$ due to the massive creation of temporary objects before reaching the final string.

The challenge of this project is therefore to move beyond this naive approach by implementing an optimised interpretation. This optimisation will allow us to reach a far higher number of iterations, which would be impossible to achieve with standard string management.

1.3 Outline of the Report

This report is structured to trace the complete design and development cycle of the interpreter. First, we will present the technical objectives set as well as the general organisation of the project.

We will then detail the critical choices regarding data structures and optimised algorithms. This section explains in a simplified manner why certain technical choices were crucial to ensuring the program's performance.

Subsequently, we will present the overall software architecture, highlighting the interaction between the different modules. Finally, we will close with an experimentation and usage phase, where we will analyse the results obtained through various demonstrations and concrete use cases in 2D and 3D.

Chapter 2

Project Structure

2.1 Requirements Description

2.1.1 External File Configuration and Parsing

One of the fundamental objectives of this project is to allow users to define the rules of an L-system without any modification to the source code. While a dedicated interface is conceivable, the use of an external configuration file proves far more relevant for automating the process.

In this approach, a simple text file becomes a structured medium where all the rules and parameters of an L-system are stored. To obtain the interpretation of a specific model, it is sufficient to provide the corresponding configuration file to the program. This method separates computation logic from data definition, thus offering great freedom of experimentation.

To make this system operational, it is necessary to develop a syntactic analyser, or parser, capable of processing the provided file according to a predefined grammar. This module is responsible for reading the textual instructions, verifying their validity, and converting them into objects that can be manipulated by the computation engine.

The parser must accurately extract essential information such as the starting axiom, the rotation angle, and the production rules. This pre-processing step is indispensable for transforming a simple sequence of characters into a logical structure ready to be expanded by the generation algorithms.

2.1.2 Graphical Interpretation and Rendering Engine

The visual core of the project relies on the interpretation of L-system rules via the *Turtle Graphics* method [4]. The main challenge here is to transform recursive logic into a coherent geometric structure, whether on a plane or in space. To this end, we have developed two specific rendering engines to manage the display according to the mode chosen by the user.

2D Visualisation

For 2D visualisation, the engine uses the Java AWT library. This choice allows easy manipulation of lines and vector paths, while offering smooth management of scaling (zoom) and

panning. The 2D engine translates each `turtle` instruction into a movement in standard Cartesian coordinates (X, Y) according to a direction angle `heading` $\vec{H}$ to produce precise fractal shapes on the canvas.

3D Visualisation

For 3D visualisation, we integrated the `Java 3D` (J3D) library to break into the depth dimension. Unlike 2D mode, this engine must manage a more complex state, including not only the position (X, Y, Z) but also the `heading` $\vec{H}$, `up` $\vec{U}$, and `left` $\vec{L}$ angles.

These vectors form a local orthogonal frame that precisely defines the orientation of the `turtle` in space. The vector $\vec{H}$ represents the direction in which the `turtle` advances, while $\vec{U}$ and $\vec{L}$ serve as references for applying the pitch, roll, and yaw rotations requested by the system. By converting each L-system angle into a transformation of this frame, the engine ensures smooth and rigorous rendering.

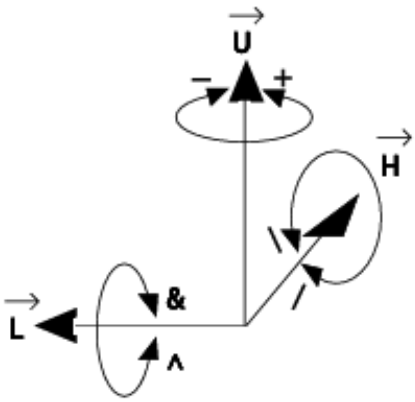

Figure 2.1: `Turtle` control in three dimensions [5]

One of the major advantages of using `Java 3D` lies in the automatic management of geometric transformations. Whereas manually computing the orientation would require the repeated application of rotation matrices to the vectors $(\vec{H}, \vec{L}, \vec{U})$, J3D allows us to implement these frame changes natively and efficiently. For readers wishing to explore the mathematical details of these transformations, Chapter 1.5 of *The Algorithmic Beauty of Plants* [5] provides a thorough analysis of these mechanisms.

Finally, the use of J3D allows free navigation around the generated object to observe it from any angle.

2.2 Implemented Features

In the context of this project, we developed and integrated the following features:

- **Configuration parser:** We designed a configuration file interpreter to dynamically load axioms, rules, and actions without modifying the source code.
- **Versatile derivation engine:** The system handles classical deterministic models but also manages **stochastic** systems, where rule selection depends on defined prob-

abilities, as well as **context-sensitive** systems, where the evolution of a symbol is determined by its immediate left and right neighbours.

- **Dual Turtle Graphics engine:** A complete implementation allows moving from simple 2D fractals to complex 3D models.
- **Optimised traversal algorithms:** Rendering uses a DFS traversal for memory efficiency and switches to a BFS algorithm to ensure the mathematical consistency of context-sensitive systems.

2.3 Project Organisation

This section details the task distribution and workflow management, from theoretical design to final implementation.

- **Arthur LETE:** Focused on the internal logic of the derivation engine (`LSystemEngine`) and the development of global data structures such as the `Symbol`, `ActionRule`, and `ProductionRule` classes. He ensured system reliability through debugging and intensive testing phases.
- **Thomas ACAR:** Responsible for the complete implementation of the `Turtle` functions (2D and 3D). He also designed the architecture of the derivation engine (`LSystemEngine`) and planned the global data structures such as the `Symbol`, `ActionRule`, and `ProductionRule` classes.
- **Zineddine GHALMI:** Developer of the `Configuration` parser for processing input files. He also designed a custom `Logger` class, facilitating user interaction and tracking of program execution flows. Furthermore, he worked jointly with Arthur on the various data structures and provided support to the other team members on various parts of the project.

Chapter 3

Technical Elements

3.1 Data Structure Description

The project architecture relies on a set of classes that separate data representation, rewriting rules, and the execution of graphical commands.

3.1.1 Symbol Management via the Flyweight Pattern:

- The `Symbol` class has no public constructor, but uses a static `Map<Character, Symbol>` named `instances`.
- This structure guarantees that a character (such as 'F' or '+') corresponds to a single unique instance in memory throughout the entire execution.
- This optimises memory usage and centralises rules: each unique `Symbol` instance stores its own list of production rules (`List<ProductionRule>`) and its associated action (`ActionRule`).

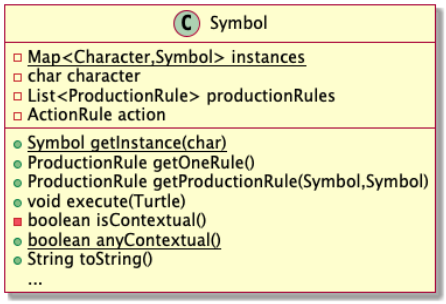

Figure 3.1: Simplified class diagram of Symbol

3.1.2 Encapsulation of Production Rules:

- The `ProductionRule` data structure encapsulates the result of a rewriting as a `List<Symbol>`, representing the successor of the rule.
- It also includes variables to manage advanced systems: a double for the application probability (which must satisfy $0 \leq p \leq 1$), as well as two characters (leftContext and rightContext) to condition the rule on its neighbourhood.

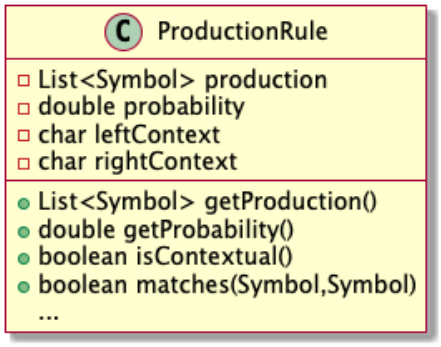

Figure 3.2: Simplified class diagram of ProductionRule

3.1.3 Mapping Symbols to Actions:

- Rather than using multiple classes for each type of movement, the ActionRule class uses Java's `Consumer<Turtle>` functional interface.
- This structure allows associating a lambda expression (such as `t -> t.turnLeft(`α`)`) with a keyword (e.g., "left", "move", "push") retrieved during the parsing of the configuration file.
- Thanks to this approach, each symbol can have its own custom `ActionRule`. This provides a level of abstraction, allowing the graphical aspect to be fully decoupled from the derivation engine.

3.1.4 The `Turtle2D` State

To model branched structures (such as trees), the system must be able to "remember" a position and orientation before creating a branch, then return to it once the branch is complete. This mechanism relies on the use of the special symbols [(push) and] (pop). When interpreting with J3D, we use the native Transform3D class to solve this problem. However, since our 2D interpreter is developed from scratch, a separate class is required to perform the same task.

The `State2D` class encapsulates the complete state of the `turtle` at a given time t:

- **Position:** Cartesian coordinates (X, Y).
- **Angle:** Current orientation in degrees.

In our rendering engine, we use a Stack<State2D> for:

- **Symbol [(Push):** The current state is pushed onto the stack (saved).
- **Symbol] (Pop):** The last state is popped and reapplied to the `turtle` (return to the branching point).

3.2 Algorithm Description

The `LSystemEngine` engine and the symbols implement several algorithms necessary to generate the fractal, ranging from stochastic selection to context-sensitive selection.

3.2.1 Stochastic Selection Algorithm (Random Drawing):

- For non-context-sensitive L-systems, the `getOneRule()` method of the Symbol class selects a production rule at random based on the assigned probabilities. It is important to note that for deterministic symbols, which have only a single production rule, this method will always return that single rule.
- The algorithm first filters out non-context-sensitive rules.
- It then generates a random value P_{rand} such that $0 \leq P_{rand} \leq 1$ using Math.random().
- It iterates over the list of filtered rules. If the value of P_{rand} is less than or equal to the probability of the current rule (P_i), that rule is selected. Otherwise, P_i is subtracted from P_{rand} and the next rule is considered.

Algorithm 1: getOneRule()

Input : A symbol s, list of rules $R = \{r_1, \ldots, r_n\}$
Output: A production rule r_k

$R_f \longleftarrow$ Filter non-context-sensitive rules of s
$P_{rand} \longleftarrow$ `Math.random` ()

for each $r \in R_f$ **do**
  if $P_{rand} \leq r.prob$ **then**
    return r
  endif
  $P_{rand} \longleftarrow P_{rand} - r.prob$
end
return r_n

3.2.2 Recursive Development Algorithm (DFS):

- For non-context-sensitive systems, the `draw` method and its auxiliary `drawRecursive` in LSystemEngine perform a depth-first traversal of the L-system derivation tree.
- At each call, if the target depth reaches 0 or if the symbol has no rule, the algorithm stops and executes the action associated with the symbol on the drawing.
- Otherwise, it derives the current symbol and calls itself recursively for each Symbol contained in the selected production rule, decrementing the depth (depth - 1).

Algorithm 2: Recursive depth-first traversal of the L-system

Input: A symbol s, current depth d

`// Base case: depth reached or terminal symbol`
if $d = 0$ ***or*** *s has no rule* **then**
  `executeAction` (s)
endif

`// Recursive case: derivation and DFS traversal`
$r \longleftarrow s$.`getOneRule` ()
foreach *symbol* $s_{child} \in r.getSymbols()$ **do**
  `drawRecursive` ($s_{child}, d - 1$)
end

This method is a cornerstone of our project. It allows symbols to be processed immediately without computing the entire last generation. Since symbols are not stored in memory (the stack unwinds as processing proceeds), the memory footprint is drastically reduced.

- **Time complexity:** $O(n^d)$ (visiting each node of the tree).
- **Space complexity:** $O(d)$ (only the current branch is kept in memory).

where n is the average number of symbols in a production rule and d is the number of generations.

By comparison, the method seen in section 1.2 would have a space complexity of $O(n^d)$, which quickly becomes unmanageable for high generation counts, whereas DFS remains limited to depth d.

The Problem of Interpreting Context-Sensitive Systems

Using a **DFS (Depth-First Search)** traversal for immediate rendering poses a major temporal synchronisation problem for context-sensitive L-systems. According to the theory of **Prusinkiewicz** in *The Algorithmic Beauty of Plants*, the derivation engine must respect the principle of ***parallel rewriting***: all symbols of a generation n must evolve simultaneously to generation $n + 1$ based on a consistent temporal state.

In DFS, a temporal offset appears: at the moment of evaluating a symbol, its left neighbour has already evolved (generation $n + 1$) while its right neighbour is still in its initial state (generation n). This lack of simultaneity makes it impossible to rigorously apply context-sensitive rules ($L < S > R$), as the immediate neighbourhood does not belong to the same temporal frame of reference.

Towards the end of the development phase, our objective was to implement context-sensitive systems. However, integrating these rules within a recursive traversal proved to be a complex algorithmic challenge.

Rather than forcing an unreliable DFS solution, we opted for a **hybrid** approach. We developed a method based on the **BFS** principle, optimised to guarantee the ***parallel rewriting*** advocated by **Prusinkiewicz**. In order not to sacrifice overall performance, our rendering engine only activates this BFS mode if the system detects the presence of context-sensitive rules. Thus, classical L-systems continue to benefit from the maximum optimisation of our recursive DFS algorithm, while complex systems benefit from mathematically exact derivation.

3.2.3 Iterative Development Algorithm with Singletons (BFS):

- Unlike classical systems, context-sensitive rules require knowledge of the state of the entire generation at time t to compute generation $t + 1$.
- The `drawForContext` method uses a `LinkedList<Symbol>` to store the initial axiom.
- For each iteration (generation), the algorithm traverses the symbols by simultaneously evaluating their left and right neighbours in order to identify a matching context-sensitive rule.
- In order to know the indices during evaluation, the future replacements are isolated in a `HashMap<Integer, List<Symbol»` where the key is the original index.

- Once the entire generation has been evaluated, the algorithm applies the replacements by sorting the keys in ascending order and computing a dynamic offset that adjusts the indices according to the size of each newly inserted string.

Unlike the naive method using `String` objects (mentioned in section 1.2), this optimised algorithm relies on the use of `LinkedList` and the `Flyweight and Singleton` pattern for symbol management. This approach considerably reduces the computational load during replacements as well as the memory footprint, providing efficient management of iterations.

Chapter 4

Project Architecture

4.1 Module and Class Diagrams

This section presents the structural organisation of our classes and modules. Please note that these diagrams prioritise architectural context clarity and abstract away secondary technical details in order to facilitate a global understanding of the project.

4.1.1 Structure of the `turtle` Package

The `turtle` package relies on a `Turtle` interface that unifies drawing commands. The `Turtle2D` engine manages 2D rendering with an association to `State2D` for stack management, while `Turtle3D` uses matrix transformations (`Transform3D`). This structure allows a direct transition between 2D and 3D rendering. See Figure 4.1.

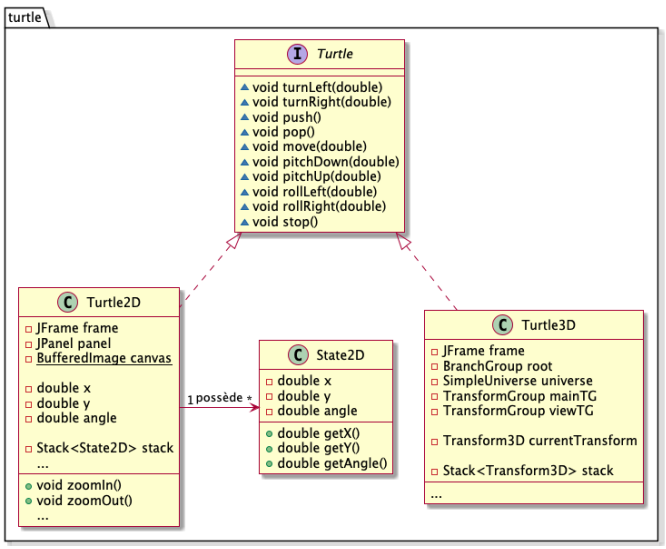

Figure 4.1: Simplified diagram of the `turtle` package

4.1.2 Structure of the `fractal` Package

This package forms the logical core of the project, grouping together the L-system derivation algorithms, the associated data structures, and the management of drawing commands. Technical details regarding the implementation of data structures are not reproduced here, as they are the subject of an in-depth analysis in section 3.1.

The architecture relies on a single instantiation of each `Symbol` per character, associating each one with a specific action and a set of production rules. The `LSystemEngine` module then exploits this structure to transform the initial axiom into a list of `Symbol` objects and generates the successive generations of the system recursively (or iteratively). See Figure 4.2.

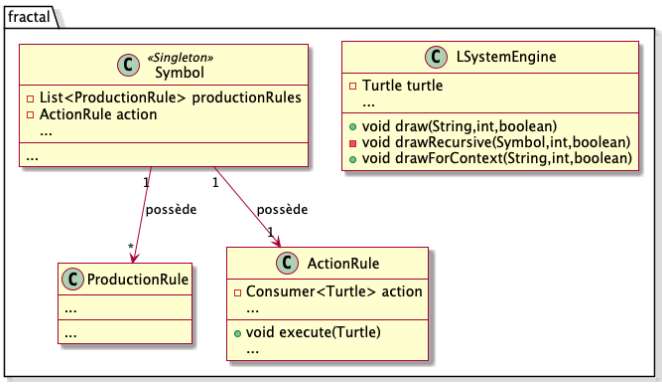

Figure 4.2: Simplified diagram of the `fractal` package

4.2 Description of Packages Used

The application relies on several standard and external libraries to provide its functionalities:

- `java.awt` & `javax.swing`: Used for designing the graphical user interface (JFrame, Panels) and managing 2D rendering on the canvas.
- `Java 3D (j3dcore, j3dutils)`: These external libraries enable the creation and manipulation of the three-dimensional graphics scene, including the management of universes and spatial transformations.
- `Vecmath`: Used for vector and matrix calculations essential to rotations and movements of the `turtle` in 3D space.
- `JogAmp (JOGL)`: Provides native bindings to OpenGL to ensure high-performance hardware acceleration when rendering complex organic structures.

4.3 Processing Pipelines

Initialisation Phase

The process begins by reading the configuration file provided by the user, which also specifies the rendering mode (2D or 3D). A `Configuration` object parses this file, validates

the syntax, and extracts the axiom as well as the rules (actions and productions). Once validation is successful, the appropriate `Turtle` object is instantiated in `main` according to the rendering mode. Then, drawing on the data extracted by the Configuration object, the system generates the unique `Symbol` objects in memory, associating each character with its respective behaviours to prepare for derivation.

Evolution and Graphical Rendering Phase

Once initialisation is complete, the `LSystemEngine` engine takes control to generate the fractal structure. Depending on the nature of the symbols, the system chooses the optimal evolution algorithm: a **BFS** (Breadth-First Search) approach for context-sensitive systems or a recursive **DFS** (Depth-First Search) approach for standard cases. During the final generation, each symbol triggers its associated action. The algorithmic details of these processes are elaborated in section 3.2.

The technical execution of these actions relies on strict delegation: the symbol passes the `Turtle` instance to its `ActionRule`. The latter executes the command (e.g., *move*, *turn*) without knowing the details of the graphical implementation. The visual aspect is entirely managed by the `Turtle`, ensuring a clear separation between the derivation logic and the final rendering. See Figure 4.3.

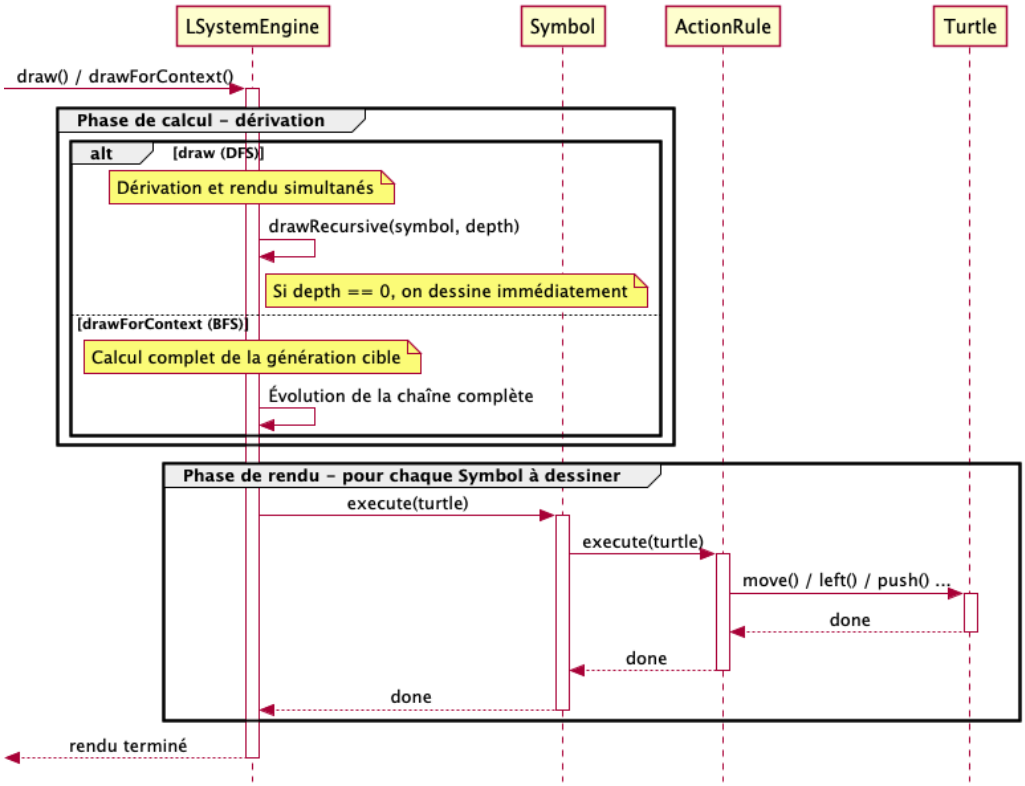

Figure 4.3: Evolution and rendering flow of the `LSystemEngine`.

By combining these two phases, we obtain the overall program structure driven by the `main` method. Consequently, the complete control flow diagram is illustrated in Figure 4.4.

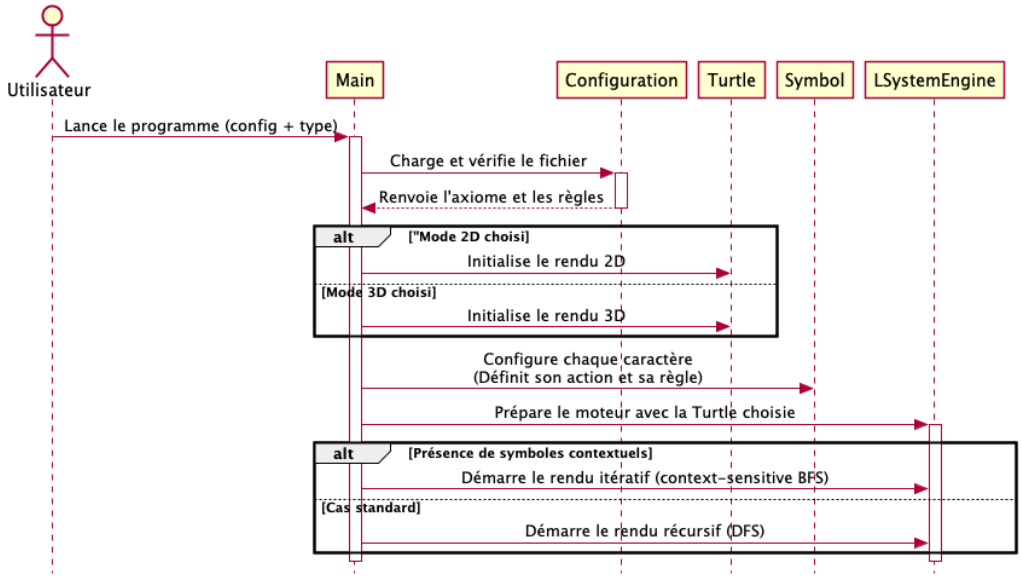

Figure 4.4: Overall control flow.

Chapter 5

Experiments and Usage

5.1 Use Cases

5.1.1 Configuration File Grammar

The configuration file is a simple text file that allows a complete L-system to be defined without modifying the source code. It is parsed by the `Configuration` class, which extracts all information required by the derivation engine.

General Structure

- Each directive occupies a single line, in the form `key = value` or `key : value`.
- **Comments** are lines beginning with `#`; they are ignored during parsing.
- **Blank lines** are ignored.
- Spaces around the separator (`=` or `:`) are tolerated.
- Keys are case-insensitive.

Table 5.1 summarises the directives recognised by the parser.

Key	Synonyms	Description
`axiom`	`axiome`	Starting string of the L-system.
`iteration`	—	Number of generations to derive.
`rule`	`regle`, `règle`	Production rule (rewriting).
`action`	—	Symbol → graphical command mapping.

Table 5.1: Directives recognised by the configuration file.

Axiom and Number of Iterations

The axiom defines the initial string from which the L-system evolves, and `iteration` sets the derivation depth:

```
axiom     = <string>
iteration = <integer>
```

Production Rules

Production rules define how a symbol is replaced at each iteration. The general syntax is:

```
rule = <symbol_specification>, <successor>[:<probability>]
```

Three forms of rules are supported:

- **Deterministic rule:** Symbol `S` is systematically replaced by its successor.

  ```
  rule = F, F-F++F-F
  ```

- **Stochastic rule:** The probability $p \in [0, 1]$ is added after the successor, separated by `:`. The sum of probabilities for all rules of the same symbol must equal 1.

  ```
  rule = F, F[+F]F[-F]F:0.33
  rule = F, F[+F]F:0.33
  rule = F, F[-F]F:0.33
  ```

- **Context-sensitive rule:** The replacement is applied only if the immediate neighbours of the symbol match the left context `L` and right context `R`. Both contexts are optional and can be used independently.

  ```
  rule = A < B > A, B       (left and right contexts)
  rule = F < W, [+L][-R]FW  (left context only)
  ```

Graphical Actions

Actions associate a symbol with a command executed by the `Turtle`. The syntax is:

```
action = <symbol>, <command>
```

Table 5.2 lists all available commands.

Command	Parameter	Description
`move`	distance	Move forward by a given distance, drawing a segment.
`left`	angle (°)	Rotate left by a given angle.
`right`	angle (°)	Rotate right by a given angle.
`up`	angle (°)	Pitch upward (3D mode only).
`down`	angle (°)	Pitch downward (3D mode only).
`rollL`	angle (°)	Roll to the left (3D mode only).
`rollR`	angle (°)	Roll to the right (3D mode only).
`push`	—	Save the current `turtle` state to the stack.
`pop`	—	Restore the last saved state from the stack.

Table 5.2: Available action commands for the `Turtle`.

Configuration File Examples

Koch Curve (deterministic, 2D):

A minimal example illustrating a classic L-system. The single rule for `F` is deterministic; Figures 1.1 and 1.2 show the generations obtained.

```
axiom = F
```

```
rule = F, F-F++F-F
iteration = 4
action = F, move 50
action = +, left 60
action = -, right 60
```

Fractal Plant (stochastic, 2D):

Three partially probabilistic rules are defined for `F`. At each iteration, one of them is chosen at random, generating a different plant at each run. The symbols `[` and `]` manage branching via a stack mechanism (*push/pop*).

```
axiom     = F
rule      = F, F[+F]F[-F]F:0.33
rule      = F, F[+F]F:0.33
rule      = F, F[-F]F:0.33
iteration = 5
action    = F, move 20
action    = +, left 25
action    = -, right 25
action    = [, push
action    = ], pop
```

Signal Propagation (context-sensitive, 2D):

The evolution of symbol `B` here depends on its left neighbour.

```
axiom = ABBBB
rule = A<B, A
rule = A, B
...
```

3D Actions:

The addition of pitch (*pitch up/down*) and roll (*roll left/right*) axes allows the system to evolve in three-dimensional space.

```
...
action = ^, up 90
action = _, down 90
action = /, rollR 90
action = \, rollL 90
...
```

5.1.2 Automated Build/Run Script (ANT)

Given the complexity of our program, the use of various libraries, and the configuration files describing fractals, we needed a tool that would allow us to compile and run the program in an automated manner while specifying certain parameters. For this purpose, we opted for the ANT tool, which perfectly meets this requirement. We wrote a complete script executed with the following command:

ant run -Dfile=<filepath> -Dturtle='2d'|'3d' -Dtext='y'|'n'

This command breaks down as follows:

- `ant run` → Full compilation and execution of the program, adding all necessary external libraries
- `-Dfile=<filepath>` → Choice of configuration file, where `<filepath>` is the path to the selected file
- `-Dturtle='2d'|'3d'` → Choice of drawing type (2D or 3D), where `'2d'|'3d'` is the selection between `'2d'` and `'3d'`
- `-Dtext='y'|'n'` → Choice of whether to save the final fractal string to a file, where `'y'|'n'` is the selection between yes and no

5.1.3 Overview of Visual Results

Once the configuration is established and execution is launched, the system generates the structures according to the defined parameters. Figures 5.1, 5.2, 5.3, 5.4 illustrate the results obtained for various models.

5.2 Quantifiable Results

As mentioned at the beginning of this report, the expected time and space complexity for the **Koch fractal** ($F \to F - F + +F - F$) follows exponential growth of type $O(n^d)$, here $O(4^d)$. We compared three distinct approaches: the "naive" algorithm mentioned in section 1.2 optimised with **StringBuilder**, our recursive structure **DFS with Singleton**, and finally the iterative **BFS system using a LinkedList of Singletons** for context-sensitive needs.

The data analysis reveals a clear divide between raw speed and memory management, particularly apparent from the 12th generation onward:

- **Naive approach with StringBuilder (Heap memory):** Up to $d = 12$, StringBuilder is extremely performant (∼2.8s). However, its nature forces it to store the entire string in RAM. At the 14th generation, memory consumption explodes to **1.25 GB**. From the 15th generation onward, the system is completely saturated and throws an `OutOfMemoryError: Java heap space`. This approach is therefore physically limited by available RAM.
- **Recursive DFS approach (Stack & Singleton):** Our recursive structure shifts the problem to the call stack. Thanks to the **Singleton** pattern, we have only 3 symbol instances (F, +, -) in memory. While StringBuilder struggles at 1.25 GB, our DFS consumes only **74 KB** for the same task. We reduced the memory footprint by a factor of **20,000**, making execution possible where other methods fail.
- **Iterative BFS approach (LinkedList & Singleton):** The iterative approach with a LinkedList of pointers shows its limits very early. Due to massive object management, computation time becomes prohibitive from $d = 8$ onward (∼5s). For $d = 12$, the theoretical execution time exceeds 35 minutes, making this method impractical for high iteration counts.

Generation	StringBuilder (Heap)	Recursive DFS (Stack)	Iterative Singleton
d=5	$\sim$ 3 ms	$\sim$ 10 ms	$\sim$ 12 ms
d=8	$\sim$ 33 ms	$\sim$ 75 ms	$\sim$ 4981 ms
d=12	$\sim$ 2874 ms	$\sim$ 4200 ms	$\sim$ 35–40 min (est.)
d=14	**1.25 GB RAM**	**74 KB RAM**	—
d=15	**OutOfMemoryError**	**Stable & Executable**	—

Table 5.3: Comparison of performance and memory consumption.

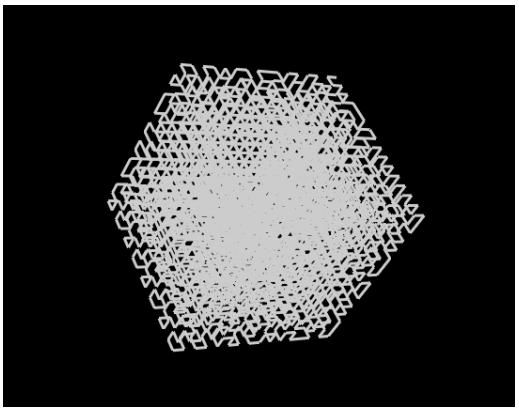

Figure 5.1: 3D visualisation of the Hilbert Curve

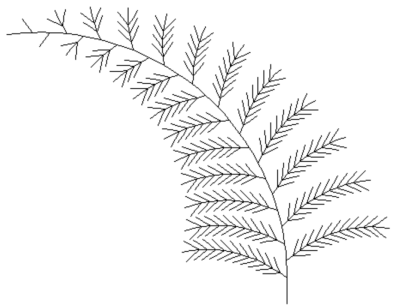

Figure 5.2: 2D visualisation of a branching system.

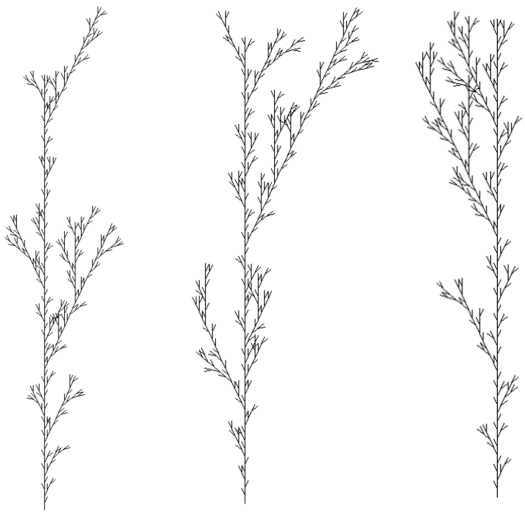

Figure 5.3: Different variants of a stochastic system.

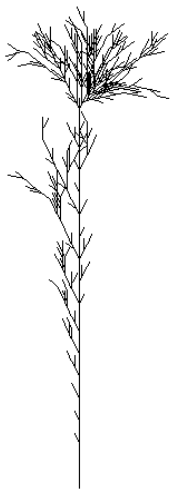

Figure 5.4: 2D visualisation of a context-sensitive system.

Chapter 6

Conclusion

This project allowed us to design and implement a Lindenmayer system (L-system) interpreter in Java, capable of modelling complex fractal structures. The developed architecture successfully meets the initial objectives: through the parsing of an external configuration file, our system effectively separates data definition from computation logic. Furthermore, the integration of a dual Turtle Graphics engine enables faithful spatial visualisation, both for 2D drawings and for 3D models.

From an algorithmic standpoint, the outcome is particularly positive with regard to complexity. The use of a recursive traversal (DFS) coupled with the Singleton pattern drastically reduced the memory footprint, enabling execution at high generation counts where a naive approach fails. To handle context-sensitive systems and ensure their mathematical consistency, we switched to an iterative algorithm based on the BFS principle.

However, our experiments highlight the limitations of this iterative BFS approach, whose computation time becomes prohibitive as the number of generations increases due to massive object management. As future improvement directions, forthcoming work should focus on optimising this iterative algorithm or on developing a recursive algorithm that allows parallel writing, in order to reduce its computational load and thus enable the generation of high-iteration context-sensitive L-systems with smooth performance.

Bibliography

[1] EPFL CS108. L-systems (lindenmayer systems). https://cs108.epfl.ch/archive/19/e/LSYS/LSYS.html. Consulté le 11 avril 2026.

[2] Oracle. String (java platform se 8). https://docs.oracle.com/javase/8/docs/api/java/lang/String.html. Consulté le 13 avril 2026.

[3] MIT OpenCourseWare. Software construction - reading 9: Immutability. https://ocw.mit.edu/ans7870/6/6.005/s16/classes/09-immutability/, 2016. Consulté le 13 avril 2026.

[4] Tao Ju, Quentin Rice, and Michelle Holloway. Turtle geometry in computer graphics and computer aided design. https://www.cs.wustl.edu/~taoju/research/TurtlesforCADRevised.pdf, 2004. Consulté le 13 avril 2026.

[5] Przemyslaw Prusinkiewicz and Aristid Lindenmayer. *The Algorithmic Beauty of Plants*. Springer-Verlag, New York, 1990. Disponible en ligne sur http://algorithmicbotany.org/papers/#abop.